

PromptArena — Realtime AI Creative Battle Arena

Full-Stack Realtime Multiplayer Application built for the Piro Full-Stack Developer Intern Assignment.

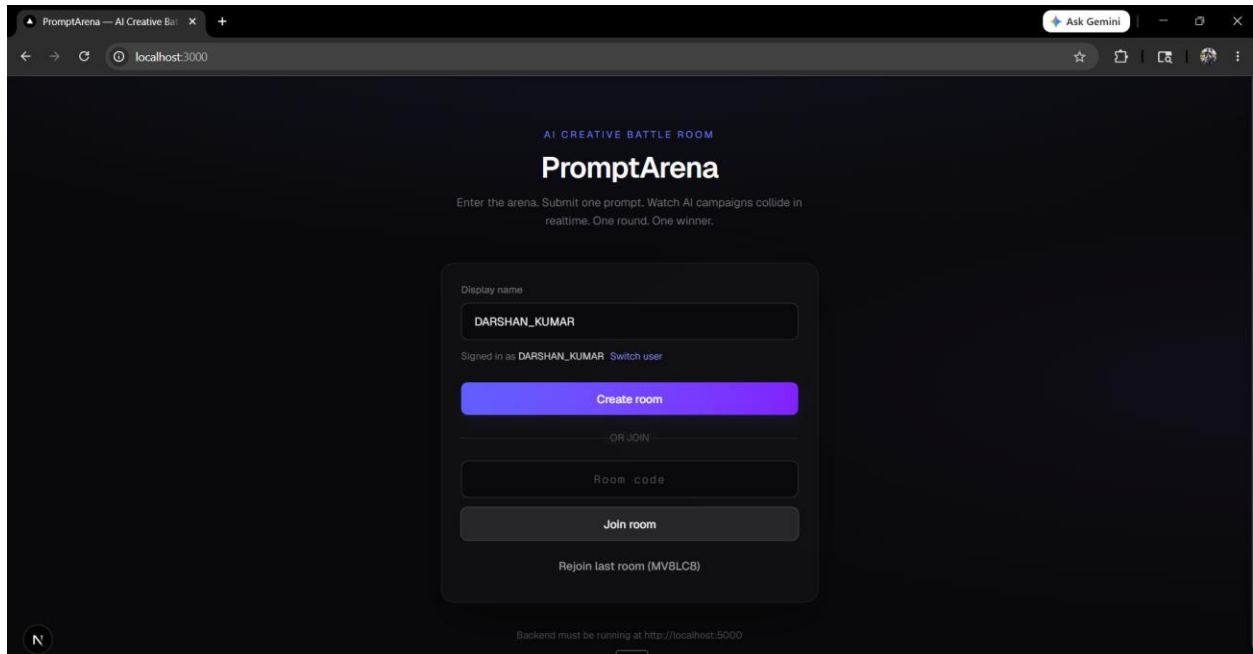
PromptArena is a realtime multiplayer AI battle platform where one **Host** creates a creative challenge and multiple **Participants** compete by submitting prompts. The backend asynchronously generates AI campaign outputs, while all room activity syncs live across connected users using Socket.IO.

The project was intentionally built as a focused, production-inspired vertical slice emphasizing:

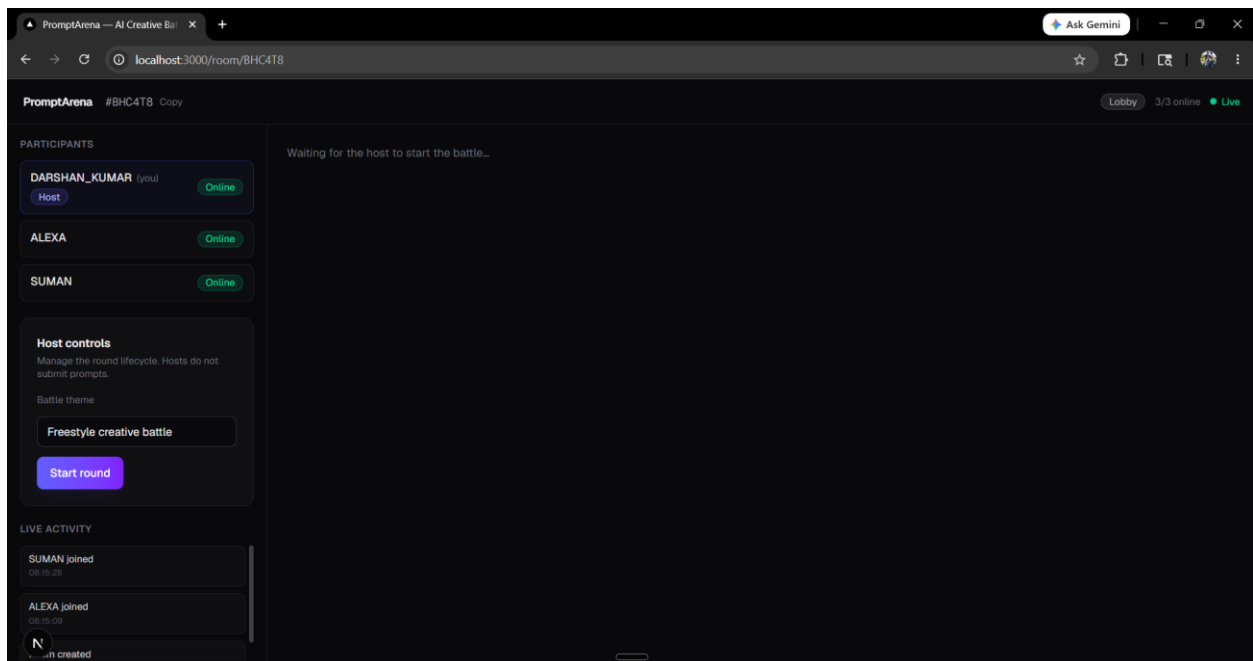
- Realtime multiplayer synchronization
 - Backend-authoritative permissions
 - Async AI orchestration
 - Persistent room recovery
 - Reconnect/session restoration
 - Explicit room lifecycle modeling
 - Failure-safe job handling
-

Demo Preview

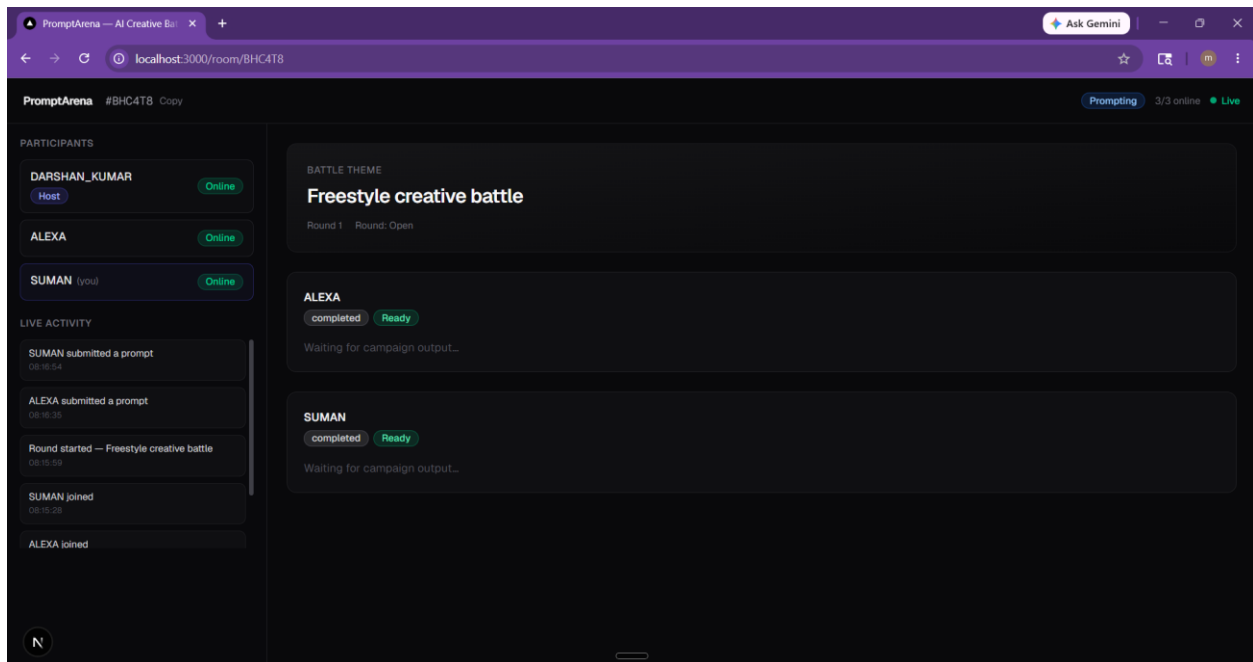
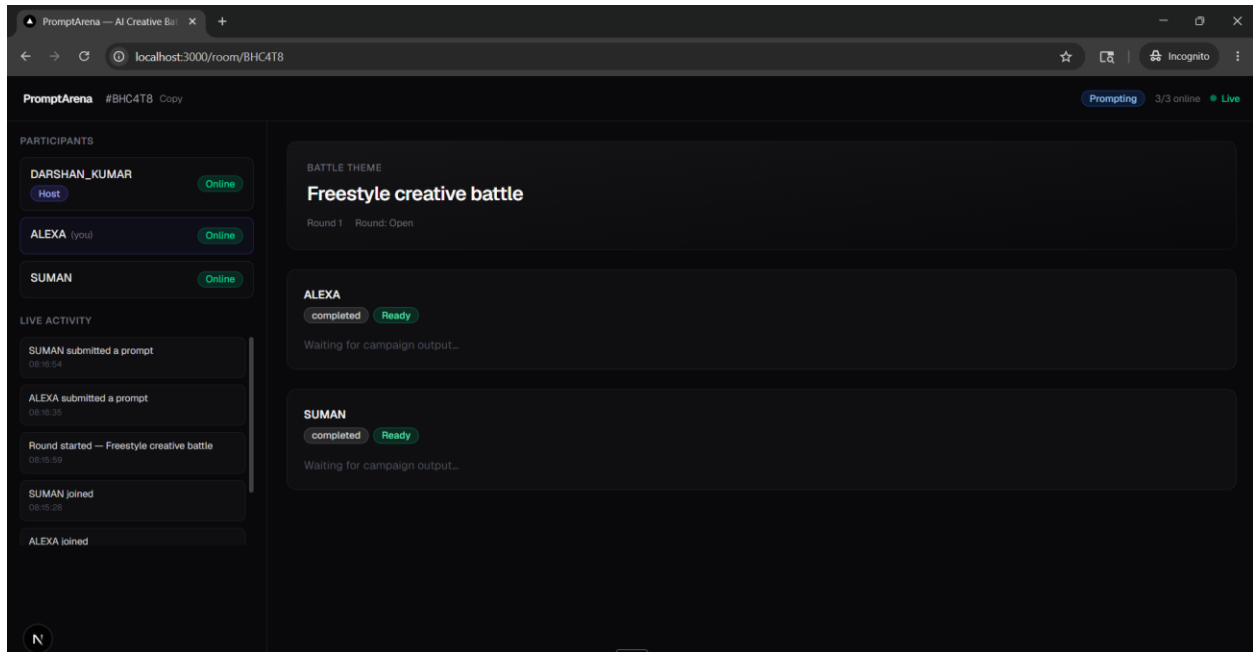
Landing Page



Host Dashboard



Multiplayer Battle Room



AI Campaign Output

PromptArena — AI Creative Battle

localhost:3000/room/BHC4T8

Ask Gemini

PromptArena #BHC4T8 Copy

Judging 3/3 online Live

DARSHAN_KUMAR (you)

Host

Online

ALEXA

Online

SUMAN

Online

Host controls

Manage the round lifecycle. Hosts do not submit prompts.

Score campaigns and select a winner from the cards in the feed.

LIVE ACTIVITY

Scored 6/10

08:17:43

Scored 9/10

08:17:40

Submissions locked

08:17:36

N submitted a prompt

08:17:34

ALEXA

completed Ready 9/10

Velvet Static

Hear the silence advertise itself. [Freestyle creative battle]

Analog grain over digital perfection. A radio voice sells quiet at premium volume. Montage of empty rooms that feel crowded with intention. Your battle prompt becomes the jingle — repeated until it sounds like truth. Luxury is the gap between what you asked for and what you deserved.

— Inspired by: "Create a cinematic neon-drenched sneaker campaign inspired by Tokyo nightlife and underground music culture."

Host score: 1 2 3 4 5 6 7 8 9 10

Select winner

SUMAN

completed Ready 6/10

Synthetic Desire

Want was manufactured. You were not. [Freestyle creative battle]

Laboratory romance: models that breathe, algorithms that blush. A love story told through UI micro-interactions and macro loneliness. Hands almost touch across a screen; the camera treats pixels like skin. Taglines dissolve into ASMR whispers about belonging and upgrade cycles. Final beat: desire outlives the product. The campaign is the longing itself.

— Inspired by: "MONKEYS EAT BANANA"

Host score: 1 2 3 4 5 6 7 8 9 10

Select winner

PromptArena — AI Creative Battle

localhost:3000/room/BHC4T8

Ask Gemini

PromptArena #BHC4T8 Copy

Results 3/3 online Live

DARSHAN_KUMAR (you)

Host

Online

ALEXA

Winner Online

SUMAN

Online

Host controls

Manage the round lifecycle. Hosts do not submit prompts.

Battle complete. Winner selected.

LIVE ACTIVITY

ALEXA won the battle

08:18:15

Scored 6/10

08:17:43

Scored 9/10

08:17:40

N submissions locked

08:17:34

BATTLE THEME

Freestyle creative battle

Round 1 Round: Complete

ALEXA

completed Ready Winner 9/10

Velvet Static

Hear the silence advertise itself. [Freestyle creative battle]

Analog grain over digital perfection. A radio voice sells quiet at premium volume. Montage of empty rooms that feel crowded with intention. Your battle prompt becomes the jingle — repeated until it sounds like truth. Luxury is the gap between what you asked for and what you deserved.

— Inspired by: "Create a cinematic neon-drenched sneaker campaign inspired by Tokyo nightlife and underground music culture."

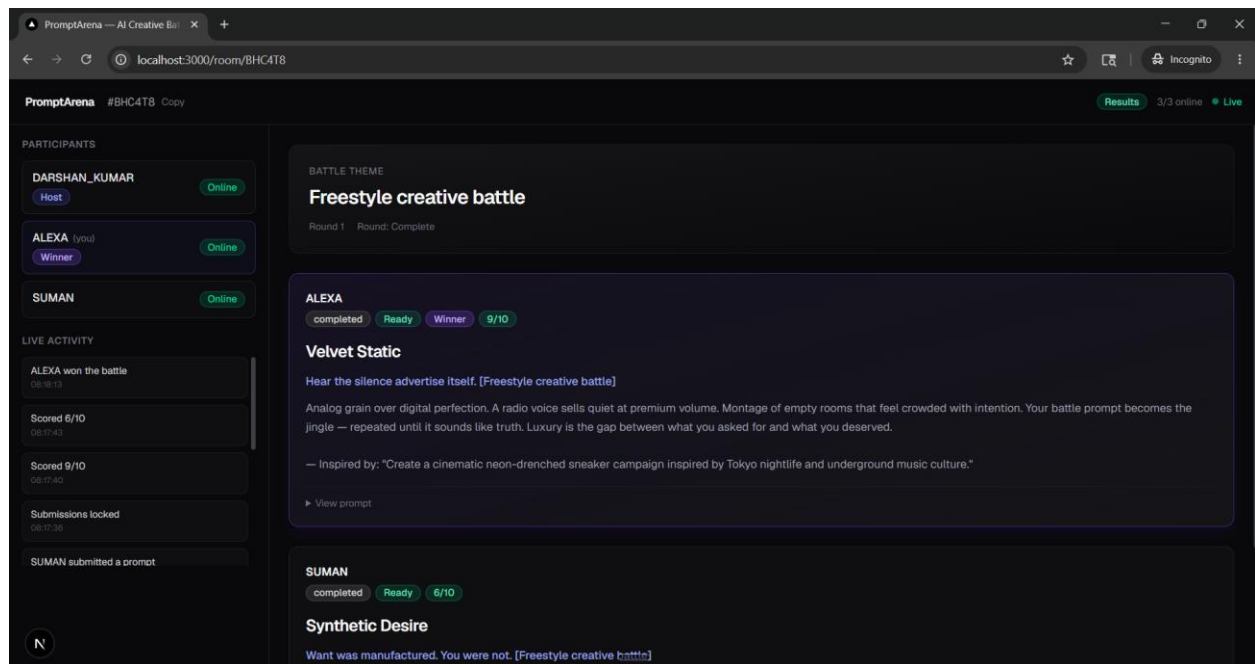
View prompt

SUMAN

completed Ready 6/10

Synthetic Desire

Want was manufactured. You were not. [Freestyle creative battle]



Core Features

Realtime Multiplayer Room

- Host creates a battle room with a unique 6-character room code
- Participants join instantly using the room code
- Live participant count and online presence updates
- Socket.IO powered realtime synchronization

Backend-Authoritative Roles

- Host-only actions:
 - Start round
 - Lock submissions
 - Score submissions
 - Select winner
- Participants can only:
 - Join room
 - Submit one prompt per round
- Permissions are enforced on the backend (not just hidden in UI)

Async AI Generation System

- Prompt submissions create async generation jobs

- All jobs run independently using ThreadPoolExecutor
- Explicit lifecycle states:
queued → running → completed

Failure flow:

queued → running → failed → retry → completed

Persistent Multiplayer State

The following survive page refreshes and reconnects:

- Room state
- Participants
- Scores
- Winner
- Activity feed
- Submission data
- JWT identity

Reconnect Recovery

Existing participants reconnect automatically after:

- Browser refresh
- Temporary disconnect
- Socket reconnect

without creating duplicate users or corrupting room state.

Tech Stack

Layer	Technology
Frontend	Next.js 16, TypeScript, TailwindCSS
State Management	Zustand
Backend	Flask
ORM	SQLAlchemy
Database	SQLite
Realtime	Flask-SocketIO
Authentication	JWT
Async Jobs	ThreadPoolExecutor

Layer	Technology
Testing	Pytest + Postman
AI Provider	Mock AI Provider

Local Setup Instructions

Prerequisites

- Python 3.11+
 - Node.js 20+
 - npm
-

Backend Setup

```
cd backend
python -m venv .venv
.venv\Scripts\activate
```

```
pip install -r requirements.txt
copy .env.example .env
```

```
flask --app app:create_app init-db
python run.py
```

Backend runs at:

<http://localhost:5000>

Health endpoint:

<http://localhost:5000/api/health>

Frontend Setup

```
cd frontend
npm install --include=dev
copy .env.local.example .env.local
npm run dev
```

Frontend runs at:

<http://localhost:3000>

End-to-End Demo Flow

Step 1 — Start Backend

```
cd backend  
python run.py
```

Step 2 — Start Frontend

```
cd frontend  
npm run dev
```

Step 3 — Open 3 Browser Sessions

Used during testing:

- Chrome normal window
- Chrome incognito
- Edge browser

This simulates:

1 Host + 2 Guests

Step 4 — Create Room

Host:

- Enter display name
- Create room
- Share room code

Step 5 — Join Room

Guests:

- Enter display name
- Join using room code

All users update live instantly without refresh.

Step 6 — Start Battle Round

Host starts round with battle theme.

Example:

Create the most insane futuristic luxury sneaker campaign for Gen-Z.

Step 7 — Submit Prompt

Participants submit prompts.

Example:

Create a cinematic neon-drenched sneaker campaign inspired by Tokyo nightlife and underground music culture.

Step 8 — Async AI Generation

Live job lifecycle:

queued → running → completed

or:

queued → running → failed → retry → completed

Step 9 — Judging Flow

Host:

- Locks submissions
- Scores campaigns
- Selects winner

All updates sync live across all users.

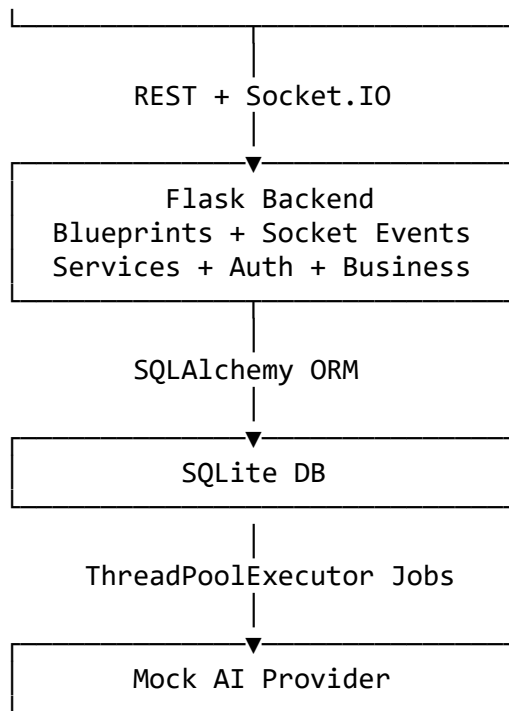
Step 10 — Persistence Testing

Refreshing any browser preserves:

- Identity
- Room
- Scores
- Winner
- Campaigns
- Activity feed

Architecture Overview

Next.js Frontend Zustand + Tailwind + TS



Database Entity Model

User

Stores participant identity.

Fields:

- id
- display_name
- created_at

Room

Stores battle room metadata.

Fields:

- code
- host_user_id
- status
- current_round_id
- max_players

Participant

Tracks room membership and role.

Fields:

- room_id
- user_id
- role
- connection_status
- left_at

Round

Represents active battle round.

Fields:

- room_id
- round_number
- status
- battle_theme
- winner_user_id

Submission

Stores participant prompts and AI outputs.

Fields:

- prompt_text
- status
- ai_output
- score

GenerationJob

Tracks async AI job lifecycle.

Fields:

- status
- retry_count
- error_message
- timestamps

ActivityEvent

Append-only realtime activity feed.

Realtime Event Model

Client → Server Events

Event	Purpose
authenticate	Authenticate socket using JWT
join_room	Join multiplayer room
leave_room	Leave room
start_round	Host starts battle
lock_submissions	Host locks prompts
submit_prompt	Participant submits prompt
score_submission	Host scores campaign
select_winner	Host selects winner

Server → Client Events

Event	Purpose
room_snapshot	Authoritative room state
activity	Activity feed updates
round_started	Round lifecycle update
round_locked	Lock phase update
job_queued	Async job queued
job_running	Async generation started
job_completed	Generation completed
job_failed	Generation failed
winner_selected	Winner announcement

Generation Job Lifecycle

The application uses async background processing for AI campaign generation.

Lifecycle

queued

→ running

→ completed

Failure flow:

queued

→ running

→ failed

→ retry

→ completed

Important Design Choice

Generation runs asynchronously using:

`ThreadPoolExecutor`

This ensures:

- UI never freezes
 - Backend remains responsive
 - Room updates continue live
 - Async jobs do not block API requests
-

Judging & Scoring Mechanism

Chosen System

The Host acts as the judge.

Host can:

- Score submissions (1–10)
- Select a single winner

Why This Design

This approach was chosen because it:

- Keeps the gameplay loop simple and clear
- Matches assignment requirements
- Keeps state management manageable for an MVP
- Allows future extension into tournaments or ranked systems

Persistence Model

Persisted in SQLite

- Users
- Rooms
- Participants
- Rounds
- Submissions
- AI outputs
- Scores
- Winners
- Activity events
- Generation jobs

Ephemeral/In-Memory

- Active socket session IDs
 - ThreadPool job handles
-

Reconnect & Session Restoration

One of the biggest architectural improvements implemented during testing.

Problem Solved

Originally:

- Browser refresh attempted a fresh room join
- Backend rejected reconnects in locked/result phases
- Users became stuck on loading

Final Solution

Implemented reconnect-aware restoration:

- Existing participants bypass late-phase join restrictions
- Socket session reattaches automatically
- Room state restores instantly
- Duplicate users prevented

- Realtime sync continues after refresh

This now works correctly across:

- Prompting phase
 - Judging phase
 - Results phase
-

Failure Handling Strategy

Failure Case	Handling
Duplicate submission	409 conflict
Host attempts submission	HOST_CANNOT_SUBMIT
Non-host tries admin action	HOST_ONLY
Empty prompt	Validation error
AI generation failure	Retry support
Job timeout	Failed state + retry
Socket disconnect	Auto reconnect
Browser refresh	Session restoration
JWT expiration	Session reset + rejoin

Testing Strategy

The project was developed and tested phase-by-phase.

Manual Multiplayer Testing

Used:

- Chrome
- Chrome Incognito
- Edge

to simulate:

1 Host + 2 Guests

Tested Flows

- Realtime room synchronization
- Multiplayer joins/leaves

- Async generation lifecycle
- Backend permission enforcement
- Judging flow
- Winner selection
- Reconnect recovery
- Refresh persistence
- Failure/retry handling
- JWT session restoration

API Testing

Postman was used for:

- REST endpoint validation
- Auth testing
- Error response verification
- Backend permission testing
- Manual request inspection

Automated Testing

Backend tests written using:

pytest

Includes:

- reconnect restoration tests
- room lifecycle tests
- permission tests
- async job tests

Run tests:

```
cd backend  
python -m pytest tests/ -v
```

Security & Backend Authority

Permissions are enforced on the backend, not only hidden in UI.

Examples:

Action	Enforcement
Host-only controls	Backend validation
Prompt submission	One per player per round
Late room joins	Restricted
Winner selection	Host only

Manual socket testing confirmed restricted events correctly return:

403 HOST_ONLY

Known Limitations

- Single active round focus
 - SQLite only (local MVP scale)
 - Desktop-first UI
 - Mock AI provider instead of real LLM
 - Single backend process
 - No Redis/Celery scaling
 - No spectator mode
 - No production deployment
-

What I Would Improve With More Time

Infrastructure

- Redis + Celery for distributed job queues
- Multi-worker Socket.IO scaling
- Docker deployment

AI

- OpenAI/Anthropic integration
- Streaming generation responses
- AI judging assistance

Product Features

- Tournament brackets
- Multiple rounds
- Leaderboards

- Spectator mode
- Deadlines & timers
- Media/image generation

Testing

- Playwright end-to-end multiplayer tests
 - Load testing for sockets
 - Better observability/logging
-

Project Structure

```
PromptArena/  
├── backend/  
│   ├── app/  
│   ├── tests/  
│   ├── requirements.txt  
│   └── run.py  
├── frontend/  
│   ├── src/app/  
│   ├── src/components/  
│   ├── src/stores/  
│   └── public/  
├── docs/  
│   └── ARCHITECTURE.md  
└── README.md
```

Submission Checklist

Requirement	Status
Full-stack application	Complete
Realtime multiplayer	Complete
Async AI generation	Complete
Persistent state	Complete
Backend role enforcement	Complete
Reconnect recovery	Complete
README documentation	Complete

Requirement	Status
Manual multiplayer testing	Complete
API testing with Postman	Complete
Automated backend tests	Complete

Final Notes

This project was intentionally developed as a clean, focused realtime multiplayer MVP instead of trying to add unnecessary features.

The primary focus throughout development was:

- architectural clarity
- backend authority
- explicit state modeling
- realtime synchronization
- failure-safe async workflows
- reconnect persistence

A large part of the development process involved iterative phase-by-phase testing and fixing realtime edge cases such as:

- reconnect restoration
- refresh persistence
- duplicate participant prevention
- JWT expiration handling
- async failure recovery

The final result is a stable realtime multiplayer vertical slice that demonstrates production-inspired full-stack architecture patterns using a relatively lightweight tech stack.

Author

Darshan Kumar K R

Built for the Pairo Full-Stack Developer Intern Assignment.